

协作者验证包 v1 • 四个验证任务

2026-09-25 | 面向协作者 | 目的: 用一周时间验证 4 个技术假设, 验证结果回来后我们再写正式规划

开发仓库: data-intelligence-platform (平台) | 只读参考: sql-lineage-mvp (数据智能内核, 已冻结)

0. 先读这些 (按顺序, 约 40 分钟)

1. 《项目规划与功能设计 v2》—— 边界、阶段、不做什么 (内核仓库 docs/platform-vision/)
2. 《设计说明书 v1.0》—— 页面、交互、设计令牌、组件清单
3. 《架构讨论: Harness + 插件化》§9 技术选型与契约 (已定) + 架构图 v3 —— 本次任务的依据
4. 本包 (任务卡 + 上手 + 验收)

一句话目标: 对话即入口, 回答必带凭证 (可追溯), 以及"加能力 = 加 skill, 内核不动".

1. 共同规则 (四个任务都适用)

规则	说明
独立可完成	四个任务互不依赖, 不需要等别人; 不要顺手改别人的范围
冻结资产只调用、不修改	内核仓库、integrations/lineage-client、packages/dip-contracts、审计库按现有接口复用 (见附录)。注意: 内核本身不是只读的——它含 /generate/* 生成类端点, /analyze 还会落盘 HTML 报告; "只读"只是我们客户端的调用约束, 任何基于"内核只读"的假设都要复核 (验证报告 P2-9)
一个任务一个 PR	分支 feat/tN-简述; PR 必须写清"怎么验证" (命令 + 期望输出)
门禁必须绿	bash ops/gate.sh → GATE: ALL_PASS (含 ruff + pytest + 分层守卫); 红了不许提 PR
不许提交密钥	DeepSeek Key 等放 .env (已在 .gitignore); 仓库里只留 .env.example
事实不许编	任何结论必须能指到来源 (文件/端点/行号); 拿不到就明说"没查到"
时间盒一周	到期没跑通不要硬撑: 交"卡在哪 + 已获得的证据 + 需要什么支持"

2. 任务卡

T1 把 DeepSeek Harness (dsh) 跑起来

项	内容
验证什么	这个开源 harness 能不能在我们的环境真跑起来、能不能当插件运行时与 Web 壳
交付物	① docs/notes/dsh-setup.md：从零到能对话的完整步骤（含踩坑与解决办法）② 版本锁定写法（不追 latest）③ 一张"能对话"的截图或录屏
边界	不要做鉴权、多用户、生产部署；不要改 dsh 源码；不要接我们的业务 skill（那是 T2）
验收标准	新人照着 docs/notes/dsh-setup.md 能一次跑通；文档里写明 Node/pnpm 版本、dsh 版本、启动命令、端口
提示	仓库： github.com/deepseek-ai/deepseek-harness （MIT, TypeScript + Cordis, v0.1 预览版）

T2 把一个业务 skill 经 MCP 被外壳调用

项	内容
验证什么	"万物皆插件"能不能落地：我们的能力包成 MCP skill 后，能否被 dsh 这类外壳无差别调用
交付物	① 一个 MCP skill 服务，包装现成的血缘解析（内核 <code>POST /analyze</code> ，只读）② 该 skill 的契约声明（按《架构讨论》§3 的 9 类字段：name+version / when / input_schema+output_schema / side_effect / auth_scope / idempotent+timeout+retry / evidence_kind / renderer / budget）③ docs/notes/mcp-skill.md：怎么注册、怎么被调用、返回长什么样
边界	只包装 1 个 skill，不要批量包装；不要碰写操作类能力（建表/调度那些留给后续）
验收标准	给一句「ads.ads_产销存月报 的产量怎么来的？」能返回结构化结果 + 回执（来源端点、耗时、证据条数）； <code>bash ops/gate.sh</code> 绿；有测试用例（可用已录制的真实响应做假内核）
提示	已录制的真实内核响应 fixture： <code>integrations/lineage-client/tests/fixtures/kernel-probe-2026-09-23.json</code> ；内核服务起法见 §3

T3 模型网关（薄层）+ 出口事实校验

项	内容
验证什么	① 能稳定接 DeepSeek API ② "模型不许编"这件事能用机制做到（不是靠提示词）
交付物	① 薄模型网关：统一 OpenAI 兼容入口（换模型/限流/记账不改业务）② 出口事实校验器：只允许用回执里出现过的表名、字段名、数字组答案；越界即拦截并标记 ③ 测试：构造"模型编了一个不存在的表名/数字"的输入，断言被拦截
边界	不要做提示词工程调优；不要做多轮对话；只做"网关 + 校验器 + 测试"
验收标准	拦截测试通过（给出反例与正例各 3 个）；网关能在不改进业务代码的前提下换模型（给一段配置即可切换）； <code>bash ops/gate.sh</code> 绿

提示	校验器可以独立于模型实现（入参：模型输出文本 + 回执白名单），这样最好测
----	---------------------------------------

T4 智能防火墙最小版（独立服务）

项	内容
验证什么	三档判定 + 一次性令牌 + 全量留痕，能不能独立成服务并真拦住危险动作
交付物	① 独立服务（单独的进程/容器，提供 HTTP 接口：判定 与 校验令牌）② 策略表（可配置：动作 → 档位）③ 判定与审批落审计库（复用现有 dip-pg）④ docs/notes/firewall.md：三档定义、策略怎么改
边界	不要做审批流程引擎/多人流转（P1 只保留"上一级确认一次"）；不要接入真实调度平台写操作
验收标准	① truncate table x / drop table x → 拒绝；② 生产写（如"上线工作流"）→ 需审批；③ 只读查询 → 自动通过；④ 令牌不可复用（用两次第二次被拒）；⑤ 四次判定全部在审计库里能查到（含判定依据与审批人）
提示	三档定义见《架构讨论》§2.2 与架构图 v3 的判定表；跨环境动作并入"需审批"档

3. 上手说明（环境）

```
# 现有可复用服务（都在本机，幂等脚本）
bash ops/start-pg.sh # 审计库 PostgreSQL :15432（会话/审计已能用）
bash /usr/local/bin/start-lineage-api.sh # 数据智能内核 :18080（脚本位置：内核仓库 ops/start-lineage-api.sh）
# 首次使用要先构建演示数据（否则口径为空）：
# cd <内核仓库> && PYTHONPATH=.:apps/lineage-api/packages/lineage-core python -m lineage.cli kb build examples/warehouse example
# Windows 下脚本是 .venv/Scripts/，且内核启动脚本需自行补 PYTHONPATH（验证报告 P1-4/P1-5b）
bash ops/start-portal.sh # 平台后端 + 前端 :18100（http://127.0.0.1:18100/app/）

# 开发
make venv # 建虚拟环境（python3.10；缺 ensurepip 先 apt install python3.10-venv）
bash ops/gate.sh # 一键验收：lint + 测试（提交前必须绿）
bash ops/gate.sh --with-smoke # 带上真内核的冒烟测试
```

代码结构（data-intelligence-platform）：apps/portal-api（后端）、apps/portal-web（前端）、packages/dip-contracts（契约层）、packages/dip-agent（编排）、integrations/lineage-client（内核客户端）、docs/adr（决策记录）。

分层铁律：依赖只能自上而下（有测试强制），契约层不许引重量级依赖。

.env.example 建议内容（新任务如需要新建）：

```
DEEPSEEK_API_KEY=sk-xxx # 不要提交真实值
DEEPSEEK_BASE_URL=https://api.deepseek.com
LLM_MODEL=deepseek-chat
DIP_PG_DSN=postgresql://dip:dip@127.0.0.1:15432/dip
LINEAGE_BASE_URL=http://127.0.0.1:18080
```

4. 验收方式（谁验、怎么验）

验收人：项目负责人（Hermes 代跑）。流程：协作者提 PR → 我跑门禁 + **按任务卡的验收标准逐条执行** → 逐条给"通过/不通过 + 证据" → 全通过才合并，否则打回并写明原因。

逐条存证要求（验收只看这些）：

- 1. 命令 + 原始输出（贴在本包 \$5 的回报表里，或存 docs/evidence/tN-*.txt|png）
- 2. 测试文件路径 + `pytest` 通过输出
- 3. 关键行为的反例（如 T3 的"编表名被拦"、T4 的"truncate 被拒"）

5. 回报格式与节奏

一周一轮；每周一次 15 分钟同步（进展 / 卡点 / 下一步）。回报按这张表填：

任务	状态	证据（命令/截图/PR）	卡点	需要什么支持
T1 dsh 起壳				
T2 MCP skill				
T3 网关 + 出口校验				
T4 防火墙最小版				

回报纪律：说"完成了"必须带证据；没完成就说卡在哪（我们宁要诚实的中途报告，不要漂亮的假完成）。

中文 Windows 的坑（务必记住）：做 HTTP 验证时必须显式 UTF-8——

`Content-Type: application/json; charset=utf-8` + 请求体用 UTF-8 字节。

否则 PowerShell 会按 GBK 发送，中文问题变成乱码，**会被误判成产品缺陷**（验证报告 \$8 就踩过这个坑）。

6. 附录：当前可复用资产（只调用，不修改）

资产	位置 / 用法	能力
数据智能内核	<code>:18080</code> ，契约 17 端点 (<code>/analyze</code> 、 <code>/upstream</code> 、 <code>/impact</code> 、 <code>/kb/*</code> 、 <code>/report</code>)	SQL 血缘解析、业务口径检索、影响面、HTML 报告
知识库（结构化）	<code>/kb/search</code> 、 <code>/kb/metric</code> 、 <code>/kb/summary</code>	口径 75 / 字段 298 / 术语 139 / 规则 76（只读）
内核客户端	<code>integrations/lineage-client</code>	参数归一化、 <code>success</code> 判定、超时重试、失败不抛异常

契约层	<code>packages/dip-contracts</code>	统一结果模型 + <code>status</code> 判定（铁律：无凭证不出结论）
编排（v1）	<code>packages/dip-agent</code>	实体识别/意图/计划/证据装配/答案成型（v2 将迁到 <code>skill + harness</code> ）
审计与会话	<code>dip-pg :15432</code> + <code>/api/audit</code> 、 <code>/api/sessions</code>	每次问答留痕：问题/调用/结论/状态/耗时
前端（静态迭代版）	<code>:18100/app/</code>	对话 + 证据面板（口径/血缘/审计）已跑通
真实响应 fixture	<code>.../fixtures/kernel-probe-2026-09-23.json</code> （87 KB）	无内核也能写测试（不要真编造数据）

7. 明确不在本轮范围（避免范围爆炸）

用户与权限管理、审批流转引擎、调度平台写操作、DataWorks/Airflow 适配、知识库写入通道与审核流、性能优化、多环境与多租户。

—— 这些等验证结果回来、写完《v2 项目规划 v1》再排期。

8. 验证反馈处置（2026-09-25 首轮验证报告）

协作者在全新 Windows 机器上从零验证后提交了《重新验证报告》，据此修复如下（已并入本仓库）：

编号	缺陷	处置
P0-1	<code>pyproject.toml</code> 缺 <code>[build-system]</code> ， <code>pip install -e ".[api,dev]"</code> 必败（make venv / CI 全挂）	补 <code>[build-system]</code> + <code>[tool.setuptools] packages=[]</code>
P0-2	<code>psycpg</code> 被顶层 <code>import</code> 但未声明	加入 <code>api extra</code>
P0-3	建表在 import 期执行，PG 不可达时 <code>libpq</code> 无限等待 → pytest/门禁静默挂死	移到 FastAPI <code>lifespan</code> ；默认 DSN 强制带 <code>connect_timeout</code> （双保险）
P1-4	<code>ops/gate.sh</code> 硬编码 <code>.venv/bin/</code> ，Windows 必失败	自动探测 <code>.venv/Scripts</code> 与 <code>.venv/bin</code>
P1-6	冒烟测试的 demo SQL 默认指向作者家目录	改为环境变量（缺失则跳过并给指引），支持自动探测
P2-8~13	路径描述 / 内核不只读 / 契约字段漏项 / fixture 大小 / 文档落点	本文件已逐条更正； <code>docs/notes/</code> 未建，T1 的说明请落在 <code>docs/collaboration/notes/</code>

教训（已写进协作规范）：中文 Windows 下做 HTTP 验证必须显式 UTF-8，否则编码问题会被误判为产品缺陷。